

Minecraft Anvil 格式文件

08/03/2024

目录

Chapter 1	Anvil 格式.....	2
1.1	区块的种类.....	2
1.2	区块数据的存储.....	2
1.2.1	Anvil 格式.....	2
1.2.2	8KiB 区	2
1.2.3	区块数据	3

Chapter 1 Anvil 格式

1.1 区块的种类

在《Minecraft》中，游戏的进本单位是方块，游戏将这些数据存储称为区块的结构中，每个区块的大小为 16×16 方块，高度为 n 方块。区块被进一步地组织到地图块中，一个地图块包含 32×32 个区块

1.2 区块数据的存储

在 1.2 版本之后，《Minecraft》以 Anvil 格式存储，即 mca 文件。它存储的是地图块规范化后的数据，游戏运行时会加载需要的 mca 文件以解析出地图块。然后提取出需要的区块进行实时加载渲染。

1.2.1 Anvil 格式

Anvil 格式用于指导如何存储一个地图块数据。每个 mca 文件都是 Anvil 格式，并且只存储一个地图块，其文件名指出了该文件对应的地图块坐标 (r.x.z.mca)。

在 Anvil 格式中，前 8KiB 被称为 8KiB 区，它用于存储区块的索引数据和区块最后修改时间，剩余部分是用于存储区块已压缩数据的多个扇区。每个扇区的大小为 4096 字节，一个区块可以占用多个扇区。

因此 mca 文件大小一定为 4096 字节的倍数，Minecraft 不接受非 4096 字节的倍数的 mca 文件。

1.2.2 8KiB 区

8KiB 区分为两个 4KiB 区，其中前者提供区块的索引相关数据，后者提供区块最后修改的时间戳。

两个 4KiB 区都是以 4 字节为单位进行分割的，其分别存储 $[x, z]$ 区块的偏移量和 $[x, z]$ 区块的最后修改时间。

注意 x, z 从 0 开始遍历，且先遍历 x ，再遍历 z ：

$[0,0], [1,0], [2,0], \dots, [31,0], [31,1], \dots, [31,31]$

前 4KiB 区每组的前 3 字节为偏移量，以大端序存储，单位为 4096 字节，后 1 字节为该区块所用的扇区数量：

Byte	1	2	3	4
描述	偏移量			扇区数量

要计算 $[x, z]$ 区块在前后 4KiB 区中的具体字节位置，可通过如下公式得到：

$$4 \times ((x \bmod 32) + (z \bmod 32) \times 32)$$

在编程中，如 Java/C/C++ 可能会出现负数，所以需要使用 AND 运算符而不是取模来进行计算：

$$4 * ((x \& 31) + (z \& 31) * 32)$$

例如，区块 $[3, 12]$ 计算后为 1548 字节，即该区块的偏移量在前 4KiB 区的第 1548 字节的位置。最有以大端序读取偏移量，并将结果乘以 4096 即可得到该区块在 mca 文件中的字节位置，然后根据其扇区数量读取对应的扇区。

到这并没有真正获取到区块的数据，只是获取了该区块的压缩数据。

最后修改时间同理，在后 4KiB 的第 1548 字节处。

1.2.3 区块数据

在上一节中我们获取了指定区块在文件中对应的扇区位置，在每个区块第一个扇区的起始位置，有 5 个字节用于记录该区块压缩数据的信息，前 4 个字节存储有效数据的长度（单位为字节），后 1 个字节存储压缩类型标识符：

Byte	1	2	3	4	5
描述	有效数据长度				压缩类型

目前压缩类型定义了三种方案：

标识符（十进制）	方案
1	GZip（RFC1952）（未使用）
2	Zlib（RFC1950）
3 (1.15.1 之前的版本)	未压缩（未使用）

当获取到有效数据长度时，可以提取其对应的压缩数据，并根据压缩类型对这段数据进行解压，即可得到 **NBT** 的二进制格式。

注意这里指的是原始二进制格式，而不是解析后的 **SNBT** 格式或解析后的树状 **NBT** 结构。